

Week 1: AWS Scaffolding & Foundational Infrastructure (Phase 0)

Strategic Goal: Provision all underlying cloud primitives, security boundaries, networking subnets, message queues, and persistent data stores so that subsequent application deployments face zero infrastructure blockers.

Day 1: VPC & Network Scaffolding

- Provision AWS VPC in us-east-1 spanning 2 Availability Zones (us-east-1a, us-east-1b).
- Create 2 Public Subnets (for Application Load Balancers) and 2 Private Subnets (for ECS Fargate containers and ElastiCache Redis).
- Configure 1 Internet Gateway (IGW) attached to public route tables.
- Provision 1 Managed NAT Gateway with an Elastic IP in a public subnet to provide private containers outbound internet access for external scraping and LLM API queries.
- Create granular Security Groups:
 - alb-sg: Allows inbound HTTPS (port 443) and HTTP (port 80) from 0.0.0.0/0.
 - ecs-api-sg: Allows inbound traffic on port 5000 strictly from alb-sg.
 - ecs-worker-sg: Egress only; no inbound ports (workers pull tasks from SQS).
 - redis-sg: Allows inbound port 6379 strictly from ecs-api-sg and ecs-worker-sg.

Day 2: IAM Security Boundaries & Role Creation

- Define least-privilege IAM execution roles:
 - ecsTaskExecutionRole: Allows ECS to pull Docker images from AWS ECR and write logs to CloudWatch Log Groups.
 - apiServerRole: Grants permissions to upload to S3, publish messages to SQS, read/write DynamoDB, and read Secrets Manager.
 - workerTaskRole: Grants permissions to poll/delete messages from SQS, stream CSVs to S3, and publish status packets to Redis.
 - lambdaWebsocketRole: Allows Lambda to manage active WebSocket connection IDs via ExecuteAPI:ManageConnections.

Day 3: Storage Buckets & DynamoDB State Tables

- S3 Data Bucket (align-ai-data-production):
 - Configure default SSE-S3 encryption and block all direct public access.
 - Establish prefix structure:
 - /uploads/: Candidate resume PDFs.
 - /scrapes/: Raw and normalized job CSVs.
 - /finder/: Smart Finder JSON dossiers.
 - /align/: Claude reasoning match results.
 - /shared/knowledge_base/: nutraceutical_kb.json and taxonomy reference files.

- S3 Static Bucket (align-ai-static-frontend): Configured for static asset delivery via CloudFront.
- Amazon DynamoDB Tables (On-Demand Capacity):
 - align_tasks (Partition Key: task_id [String]) — Stores state (queued, running, completed, failed), error traces, and output S3 keys.
 - api_costs (Partition Key: request_id [String], Sort Key: timestamp [Number]) — Stores token counts and dollar spend per LLM call.
 - kb_progress (Partition Key: company_name [String]) — Tracks offline research indexing progress.

Day 4: Message Queues, Redis Cluster & Secrets Manager Vault

- Amazon SQS Queues:
 - scraper-jobs (Visibility timeout: 30 mins, Message retention: 4 days).
 - finder-jobs (Visibility timeout: 60 mins, Message retention: 4 days).
 - align-jobs (Visibility timeout: 20 mins, Message retention: 4 days).
 - failed-jobs-dlq (Dead-Letter Queue capturing messages that fail 3 consecutive attempts).
- Amazon ElastiCache Redis: Launch single-node cache.t3.micro cluster in private subnets for real-time pub/sub and result caching.
- AWS Secrets Manager: Create encrypted secret production/align-ai/credentials storing:
 - ANTHROPIC_API_KEY
 - OPENAI_API_KEY
 - APIFY_TOKEN
 - THEIRSTACK_TOKEN
 - SERPER_API_KEY
 - CHROMADB_API_KEY

Day 5: Automated Verification & Smoke Testing

- Execute a standalone Python verification script using boto3 from local development environment to test:
 1. Uploading and downloading objects in S3.
 2. Writing, querying, and updating items in DynamoDB.
 3. Sending and polling messages in SQS queues.
 4. Fetching secrets from Secrets Manager.

Deliverables & Quality Gates:

- Infrastructure-as-Code (Terraform / CloudFormation) versioned in repository.
 - S3 bucket policies enforce server-side encryption and block public ACLs.
 - Smoke test script passes with 100% success across all provisioned services.
-

Week 2: Stateless API Server & Frontend Migration (Phase 1)

Strategic Goal: Refactor app.py from a stateful monolith into a stateless REST API, containerize it on ECS Fargate behind an Application Load Balancer, and decouple the frontend via CloudFront CDN.

Targeted Files for Modification:

- job-aggregator-final/app.py (25 routes refactored)
- job-aggregator-final/templates/index.html
- job-aggregator-final/templates/finder.html

Day 1: Code Refactoring — State Externalization

- Open job-aggregator-final/app.py.
- Delete in-memory task dictionaries: align_tasks = {}, finder_tasks = {}, and global scraping_status.
- Write services/dynamo_service.py with reusable helper methods:
 - create_task(task_id, task_type, params)
 - get_task_status(task_id)
 - update_task_status(task_id, status, result_key=None, error=None)
- Refactor API endpoints (/api/scrape, /api/finder/start, /api/align_ai/start) to immediately record a queued state in DynamoDB and return { "task_id": "...", "status": "queued" }.

Day 2: Code Refactoring — SQS Dispatch & S3 Storage

- Write services/sqs_service.py to route job payloads to scraper-jobs, finder-jobs, or align-jobs.
- Update file upload routes to stream uploaded PDF resumes directly to s3://align-ai-data-production/uploads/{task_id}.pdf using boto3.client('s3').upload_fileobj().
- Remove local file reads (.apify_token, .theirstack_token) and replace with cached AWS Secrets Manager client.
- Deprecate legacy SSE routes (/api/logs/stream, /api/align_ai/stream/<id>, /api/finder/stream/<id>).

Day 3: Dockerization of API Core

- Author optimized multi-stage Dockerfile.api:
 - FROM python:3.11-slim
 - WORKDIR /app
 - COPY requirements.txt .
 - RUN pip install --no-cache-dir -r requirements.txt gunicorn
 - COPY . .

- EXPOSE 5000
 - CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "4", "--timeout", "60", "app:app"]
- Create AWS ECR repository align-ai-api-server, build image, and push to ECR.

Day 4: ECS Fargate Deployment & Load Balancing

- Create ECS Cluster align-ai-production.
- Create ECS Task Definition (0.5 vCPU, 1 GB RAM) configured with apiServerRole and CloudWatch log drivers.
- Provision an Application Load Balancer (ALB) across public subnets; configure Target Group pointing to container port 5000 with health check path /api/health.
- Launch ECS Service with desired_count = 2 for high availability across multi-AZ.

Day 5: Frontend Decoupling & CloudFront Setup

- Extract HTML templates (index.html, finder.html) and static assets (CSS/JS).
- Upload static files to S3 static hosting bucket.
- Create Amazon CloudFront distribution:
 - Cache Behavior /api/* → Routed to ALB (No caching, HTTPS redirect).
 - Cache Behavior /* → Routed to S3 static bucket (Cached, HTTPS redirect).

Deliverables & Quality Gates:

- CloudFront URL successfully loads UI in any browser over HTTPS.
- Triggering /api/scrape returns {"task_id": "...", "status": "queued"} in < 150ms.
- SQS queue receives task message and DynamoDB records queued status.

Week 3: Scraper Suite Containerization & Playwright (Phase 2A)

Strategic Goal: Package all 6 scraping engines into an isolated Docker container with headless Chromium/Playwright pre-installed, ensuring 100% execution compatibility on AWS Fargate.

Targeted Files for Modification:

- job-aggregator-final/indeed_zip_scraper.py
- job-aggregator-final/glassdoor_scraper.py
- job-aggregator-final/theirstack_scraper.py
- job-aggregator-final/unified_nuclear_scraper.py
- job-aggregator-final/job_scraper_v3.py
- job-aggregator-final/web-crawler-career/ (14 ATS platform extractors)

Day 1: Scraper Modules Normalization

- Audit all 6 scraping modules to ensure standardized parameter signatures:
 - Inputs: query, location, date_filter, max_results, log_fn.
 - Outputs: Normalized list of job dictionaries (Title, Company, Location, Date Posted, Link, Source, Job Description).
- Ensure all hardcoded local paths are replaced with parameter passing.

Day 2: Playwright Headless Configuration for Linux Containers

- Refactor job_scraper_v3.py and unified_nuclear_scraper.py to support headless browser execution on Linux:
 - browser = await playwright.chromium.launch(
 - headless=True,
 - args=[
 - "--no-sandbox",
 - "--disable-setuid-sandbox",
 - "--disable-dev-shm-usage",
 - "--disable-gpu",
 - "--single-process"
 -]
 -)
- Implement stealth user-agent rotation and anti-bot retry loops.

Day 3: Dockerization with Playwright Base

- Write Dockerfile.scrapers-worker:
 - FROM mcr.microsoft.com/playwright/python:v1.40.0-jammy
 - WORKDIR /app
 - COPY requirements.txt .
 - RUN pip install --no-cache-dir -r requirements.txt
 - RUN playwright install chromium
 - COPY . .
 - CMD ["python", "workers/scrapers_consumer.py"]

Day 4: Direct S3 CSV Export Integration

- Modify scraper export routines: instead of writing to local disk (output/results.csv), stream CSV directly to S3: s3://align-ai-data-production/scrapes/{task_id}.csv.
- Write services/s3_service.py function generate_presigned_download_url(s3_key, expiry=86400) to generate 24-hour download links for the UI.

Day 5: ECR Deployment & Container Testing

- Build and push align-ai-scraper-worker image to AWS ECR.

- Execute standalone test tasks on ECS to verify Apify actors, TheirStack REST API, and ATS crawlers execute cleanly within AWS IP space.

Deliverables & Quality Gates:

- Scraper container initializes Chromium headlessly without X11 or GPU errors.
 - Apify, TheirStack, and ATS crawlers harvest jobs successfully from within AWS subnets.
 - Output CSVs stream to S3 and generate valid, downloadable Pre-Signed URLs.
-

Week 4: SQS Worker Dispatch & Real-Time WebSocket Streaming (Phase 2B)

Strategic Goal: Connect the Scraper Worker to SQS and replace legacy blocking SSE loops with API Gateway WebSockets + Redis Pub/Sub so logs stream live to the browser without locking server threads.

Day 1: SQS Consumer Worker Daemon

- Write `workers/scraper_consumer.py`:
 - Continuous long-polling loop (`WaitTimeSeconds=20`) against scraper-jobs SQS queue.
 - On message receipt: decode task payload, update DynamoDB status to running.
 - Dispatch parameters to selected scraper module.
 - On success: upload CSV to S3, update DynamoDB status to completed, set `result_s3_key`, and delete message from SQS.
 - On failure: catch traceback, update DynamoDB status to failed, log error, and allow SQS DLQ handling.

Day 2: Redis Log Publisher Implementation

- Implement custom `RedisLogHandler` replacing legacy `sys.stdout = LogRedirector()`:
 - `publish_log(task_id, message, level="INFO")`
 - Payload contains `task_id`, timestamp, level, and message.
 - Publish to Redis channel `task:{task_id}:logs`.

Day 3: API Gateway WebSocket API & Lambda Relay

- Create AWS API Gateway WebSocket API (`align-ai-logs-ws`).
- Implement `$connect` and `$disconnect` Lambda handlers storing active `connection_id` mappings in DynamoDB table `websocket_connections`.
- Write `workers/log_streamer_lambda.py`:

- Subscribes to Redis channels and forwards messages to corresponding connection_id using boto3.client('apigatewaymanagementapi').post_to_connection().

Day 4: Frontend WebSocket Integration

- Update templates/index.html to replace legacy SSE EventSource with native WebSocket.
- Connect WebSocket to the WS endpoint using the task ID.
- Parse incoming JSON log data and append the message to the terminal log.

Day 5: Concurrency & Auto-Scaling Testing

- Deploy Scraper Worker as an ECS Service with Auto-Scaling based on SQS queue depth (scale out when ApproximateNumberOfMessagesVisible > 5).
- Trigger 5 concurrent scrape jobs from UI and verify that all 5 terminal windows stream independently with zero lag.

Deliverables & Quality Gates:

- Flask API server maintains 0% thread blocking during long-running scraping tasks.
- Client browser displays live terminal logs via WebSocket with < 200ms latency.
- SQS messages auto-delete upon job completion; failed jobs land in DLQ after 3 retries.

Week 5: ChromaDB Cloud & KB Automation (Phase 3)

Strategic Goal: Migrate vector storage to ChromaDB Cloud, completely remove Ollama/Gemma local LLM dependencies, and automate the offline company research pipeline via weekly EventBridge cron.

Targeted Files for Modification:

- matcher/batch_researcher.py
- matcher/knowledge_base/kb_builder.py
- matcher/claude_reasoning/retrieval_engine.py
- matcher/claude_reasoning/job_vectorizer.py
- matcher/claude_reasoning/company_vectorizer.py
- matcher/claude_reasoning/taxonomy_processor.py

Day 1: ChromaDB Cloud Client Migration

- Provision managed ChromaDB Cloud instance; store host URL and API key in Secrets Manager.

- Update vector database client across all vectorizer modules:
 - Replace `chromadb.PersistentClient(path="chroma_db/")` with cloud-native `chromadb.HttpClient`.
 - Use `CHROMADB_HOST` and `CHROMADB_API_KEY` environment variables.
- Verify collection definitions: `company_store`, `job_store`, `taxonomy_store`.

Day 2: Taxonomy & Rule Vector Ingestion

- Upload `Supplement_Brand_Reference_No_Tables.txt` (189 KB) to S3 (`/shared/knowledge_base/`).
- Execute `taxonomy_processor.py` to generate OpenAI text-embedding-3-small embeddings and populate ChromaDB Cloud `taxonomy_store`.

Day 3: Refactoring Batch Researcher (Eliminating Ollama)

- Refactor `batch_researcher.py`:
 - Remove all local Ollama/Gemma3 execution logic.
 - Implement Anthropic Claude 3.5 Haiku API for structured company extraction from raw web scrape text.
 - Replace local Excel file reader with S3 `get_object()` download stream.
 - Replace local `research_progress.json` with DynamoDB updates in `kb_progress` table.

Day 4: Scheduled ECS Task Setup

- Package `batch_researcher.py` and `kb_builder.py` into Docker container `align-ai-kb-builder`.
- Push image to ECR.
- Create AWS EventBridge Rule triggering the ECS Task on a weekly cron schedule (`cron(0 2 ? * SUN *)`) — 2:00 AM UTC every Sunday).

Day 5: Full Knowledge Base Build Run

- Manually trigger the KB Builder container.
- Verify processing of 500+ companies from Excel, generation of updated `nutraceutical_kb.json` in S3, and successful vector embedding into ChromaDB Cloud.

Deliverables & Quality Gates:

- ChromaDB Cloud collections populated with ~500 company profiles and full taxonomy.
 - Zero GPU/Ollama dependencies remaining in the entire codebase.
 - EventBridge cron schedule active and verified in AWS console.
-

Week 6: AI Matcher RAG Engine & Cloud Caching (Phase 4A)

Strategic Goal: Containerize the 4-stage Claude Haiku RAG matching pipeline on ECS and migrate the local MD5 file cache to shared Redis memory to eliminate redundant AI API costs.

Targeted Files for Modification:

- matcher/claude_reasoning/main.py
- matcher/claude_reasoning/candidate_parser.py
- matcher/claude_reasoning/context_builder.py
- matcher/claude_reasoning/matcher_llm.py
- matcher/claude_reasoning/cost_tracker.py

Day 1: Resume PDF Ingestion from S3

- Refactor candidate_parser.py:
 - Accept S3 key (uploads/{task_id}.pdf) as input.
 - Download PDF directly to in-memory io.BytesIO buffer for pdfplumber extraction.
 - Execute Claude Haiku candidate extraction.
 - If candidate past employers require research, query Serper API with Secrets Manager credentials.

Day 2: Shared Redis Result Cache

- In matcher/claude_reasoning/main.py, replace local cache/{hash}.json files with Redis lookups.
- Use cache key cache:match:{md5_hash}.
- Cache results with a 30-day TTL.
- Cached results should return in under 15ms.

Day 3: Context Builder & Cost Tracker Integration

- Verify token budget management in context_builder.py (~10k token limit).
- Refactor cost_tracker.py:
 - Replace local api_cost_log.json writes with DynamoDB writes to api_costs table.
 - Enforce strict \$10 per-job budget limit: abort task and log alert if cumulative token cost exceeds threshold.

Day 4: Containerization of Align AI Worker

- Author Dockerfile.align-worker, implement SQS consumer polling align-jobs, build, and push to ECR.
- Configure ECS Service for Align Worker.

Day 5: Benchmark Evaluation Run

- Run test matches on benchmark resumes (sample_resume.pdf, Dylan Egan.pdf).
- Verify match scores and reasoning are 100% consistent with previous local runs.

Deliverables & Quality Gates:

- Candidate parsing and Serper enrichment execute seamlessly in container environment.
 - Repeated candidate evaluations hit Redis cache in < 15ms, completely skipping Claude API charges.
 - Cost tracking records exact token counts and costs to DynamoDB.
-

Week 7: Smart Job Finder Orchestration & End-to-End Flow (Phase 4B)

Strategic Goal: Deploy the full 1,219-line Smart Finder orchestrator (resume_job_finder.py) in the cloud, coordinating concurrent multi-portal scraping with real-time AI scoring and reranking.

Targeted Files for Modification:

- job-aggregator-final/resume_job_finder.py
- job-aggregator-final/templates/finder.html

Day 1: Smart Finder Cloud Refactoring

- Refactor resume_job_finder.py:
 - Remove hardcoded local paths (os.path.join(MODULE_DIR, ...)).
 - Connect to ChromaDB Cloud and Redis logging.
 - Integrate S3 for resume downloads and aggregated job output uploads.

Day 2: Concurrent Multi-Portal Scraping Validation

- Verify ThreadPoolExecutor(max_workers=5) running inside ECS Fargate:
 - TheirStack API call
 - Indeed Apify actor
 - ZipRecruiter Apify actor
 - Glassdoor Apify actor
 - LinkedIn stealth session
- Verify domain filtering patterns (_domain_pattern), noise filters (_noise_pattern), and deduplication algorithms.

Day 3: Finder Worker ECS Service Deployment

- Package into Dockerfile.finder-worker (with Playwright included).
- Build and push image to ECR.
- Create ECS Service listening to finder-jobs SQS queue.

Day 4: Frontend Finder UI Integration

- Update templates/finder.html to connect to API Gateway WebSockets.
- Connect multi-stage UI progress bar:
 - Parsing Resume
 - Generating Search Terms
 - Scanning Portals
 - AI Ranking Fit

Day 5: Full End-to-End Pipeline Stress Test

- Upload 10 distinct candidate resumes across various nutraceutical roles (e.g., QA Specialist, VP Sales, Formulations Chemist).
- Validate full pipeline execution:
 - PDF upload
 - S3
 - SQS
 - Fargate Worker
 - Multi-portal Scrape
 - Claude Reranking
 - DynamoDB State
 - S3 Result
 - WebSocket Display

Deliverables & Quality Gates:

- Complete Smart Finder pipeline executes within 90–120 seconds in the cloud.
- Cross-portal deduplication successfully eliminates duplicate postings.
- Output JSON dossier safely stored in S3 and delivered to client browser.

Week 8: Production Hardening, Security, Load Testing & Cutover (Phase 5)

Strategic Goal: Implement comprehensive monitoring, automated alerting, enterprise security firewalls, performance load testing, and execute the final production DNS cutover.

Day 1: Observability & CloudWatch Dashboards

- Configure CloudWatch Metric Alarms with SNS Email/Slack notifications:

- SQS DLQ Message Count > 0 (Immediate engineer alert).
 - ALB 5XX Error Rate > 1% over 5 minutes.
 - ECS Container Memory/CPU Utilization > 80%.
 - Monthly Anthropic Spend approaching \$500.
- Build centralized CloudWatch Dashboard displaying live queue depths, active ECS tasks, request latency percentiles, and daily AI spend.

Day 2: Security Firewalls & Lifecycle Rules

- Attach AWS WAF (Web Application Firewall) to CloudFront distribution:
 - Enable AWS Managed Core Rule Set (CRS).
 - Enable Rate Limiting Rule (max 300 requests / 5 mins per IP).
 - SQL Injection and XSS Protection.
- Configure S3 Lifecycle Rules:
 - Auto-delete /uploads/ candidate PDFs after 30 days.
 - Auto-delete /scrapes/ temporary CSVs after 7 days.
- Enable DynamoDB TTL on align_tasks to auto-expire completed job records after 60 days.

Day 3: 25-User Concurrent Load Testing

- Run automated load tests simulating 25 concurrent users launching Smart Finder searches simultaneously.
- Verify ECS Auto-Scaling policies: worker pool dynamically scales from 2 to 8 Fargate tasks based on SQS queue depth.
- Inject deliberate worker faults (killing containers mid-scrape) to verify that SQS visibility timeouts automatically re-route tasks to healthy containers with zero data loss.

Day 4: Custom Domain, SSL & DNS Cutover

- Request ACM (AWS Certificate Manager) SSL/TLS certificate for production domain (alignai.io).
- Attach certificate to CloudFront distribution.
- Configure Amazon Route 53 Alias A/AAAA DNS records pointing to CloudFront.

Day 5: Production Go-Live & Handover

- Execute final end-to-end smoke test on production custom domain.
- Complete operational runbook and architecture documentation handover to engineering leadership.